

E-Commerce Order Analytics System

Project Report

An end-to-end Python + SQL data engineering project: synthetic data generation, cleaning & validation, SQLite loading, 16 SQL analyses ranging from basic aggregation to window functions and cohort analysis, and a menu-driven CLI reporting tool.

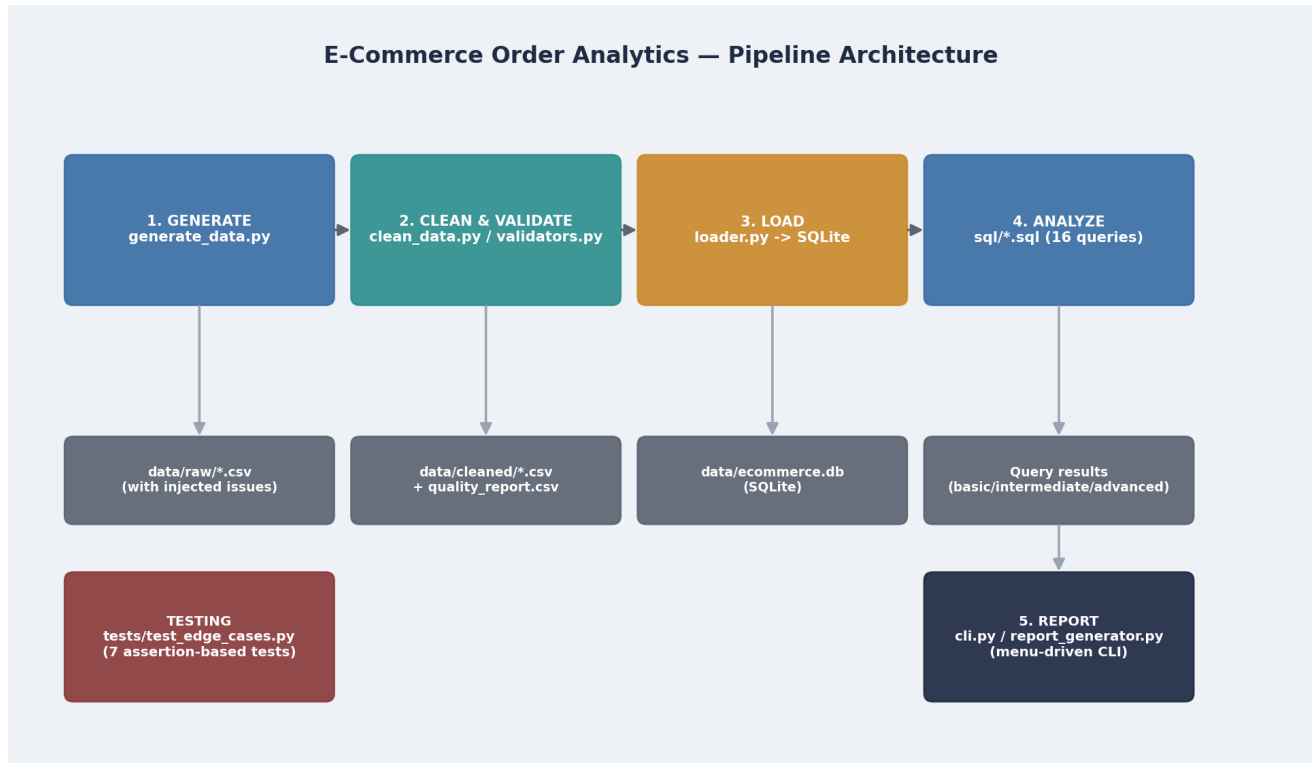


Figure 1: Pipeline architecture — generate → clean/validate → load → analyze → report.

1. Dataset Overview

Four related CSVs were generated with realistic e-commerce distributions (seasonal order spikes in Nov/Dec, a repeat-buyer segment, category-appropriate price ranges) and deliberately injected data-quality issues so the cleaning pipeline has real problems to catch.

Dataset	Rows Generated	Primary Key
customers.csv	600	customer_id
products.csv	220	product_id
orders.csv	1,500	order_id
order_items.csv	~2,800	item_id

2. Data Quality Findings

The cleaning pipeline (clean_data.py + validators.py) inspects every row against a set of explicit rules, repairing what can be safely repaired, removing rows that break referential integrity, and flagging-but-retaining rows that represent valid business events (e.g. a negative quantity is a return, not an error). Actual counts from the most recent pipeline run:

Dataset	Rule	Rows Affected
customers	invalid_email	11
order_items	orphan_order_or_product_id	34
products	messy_product_name	17
orders	missing_customer_id	84
orders	future_order_date	19
order_items	negative_quantity	84
order_items	invalid_discount_percent	42
order_items	rows_removed_orphans	34

3. SQL Analysis Coverage

16 queries were implemented and executed successfully against the loaded SQLite database, organized into three tiers of complexity:

Tier	Queries	Techniques
Basic	Revenue per category, Top 10 customers, Monthly order counts	Joins, GROUP BY, aggregation
Intermediate	Never-delivered customers, High-return products, Category performance	Conditional aggregation, HAVING
Advanced	Running totals, DENSE_RANK, LAG gap analysis, multi-level aggregation	Window functions, CTEs, self-joins, FIRST/LAST

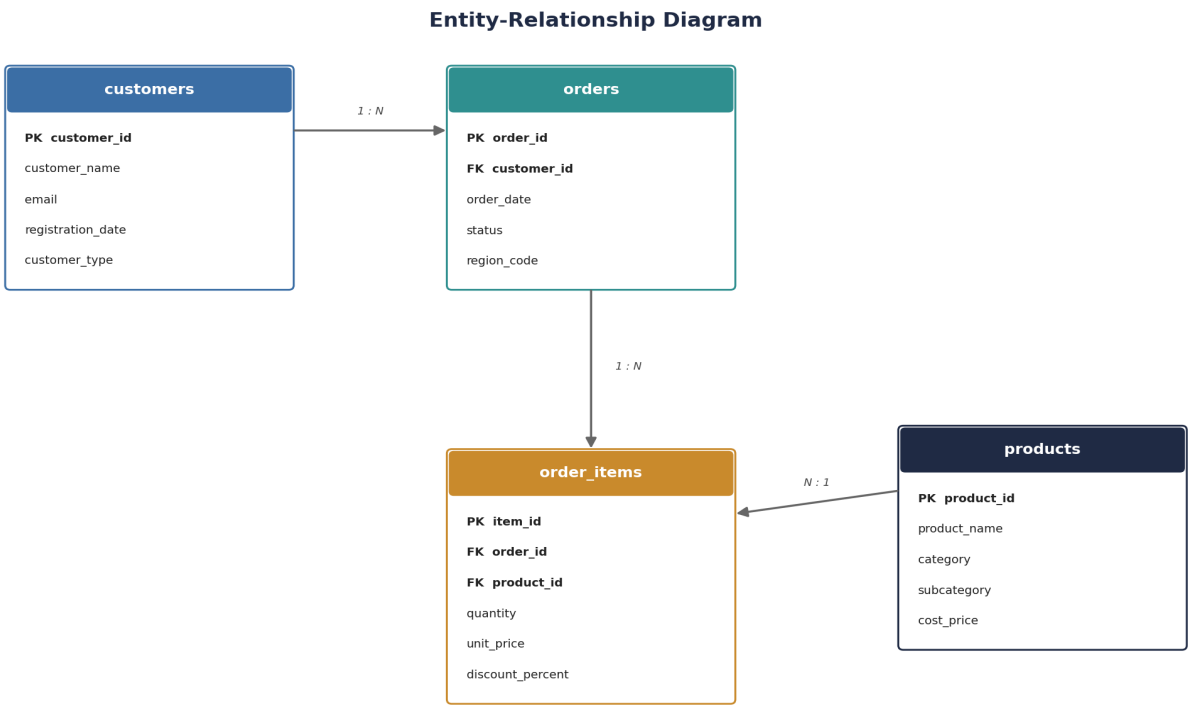


Figure 2: Entity-relationship diagram of the four core tables.

4. Edge-Case Testing

7 assertion-based tests cover the 4 required edge cases (orphaned order_id, discount_percent > 100, zero quantity, future-dated orders) plus 3 additional cases (duplicate order_id, invalid email, missing customer_id). All 7 pass on the current dataset. Results are written to data/reports/testing_report.txt on every run.

5. Lessons Learned

Referential-integrity cleanup must run against the already-deduplicated orders table, not the raw one — otherwise valid order_items referencing a de-duplicated order_id would be wrongly dropped. SQLite's FIRST_VALUE/LAST_VALUE window functions also require an explicit frame clause (ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) to behave predictably — without it, LAST_VALUE silently returns the current row instead of the partition's last row.

6. Future Improvements

- Swap SQLite for PostgreSQL and add scheduled orchestration (cron/Airflow) for refreshes.
- Add a dbt-style transformation layer so SQL is version-controlled and independently testable.
- Parameterize the CLI's date-range logic to support arbitrary (non-fixed-span) ranges.
- Add data-quality checks as a pre-load CI gate that fails the build past a defined threshold.